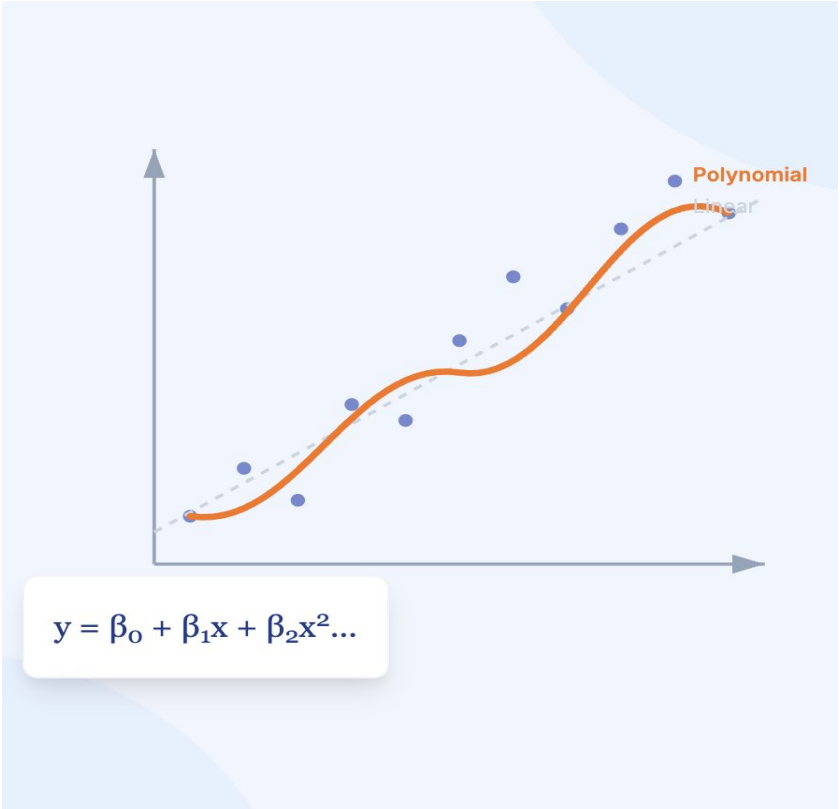
# Polynomial Regression & Regularization Techniques



# Recap - Linear Regression Works Great

- Finds the best straight line to minimize error between points.
- When data has linear relationship, linear regression is perfect.

$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + \ldots + w_nx_n$$


*Example: Predicting salary based on experience*

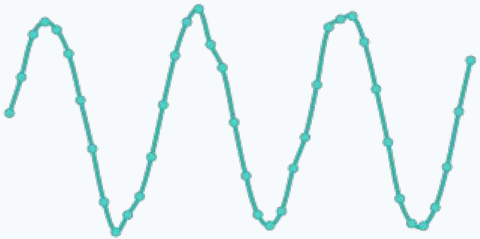
# But What About Curved Data?

**Quadratic (Parabolic)**

e.g., Projectile Motion, Gravity

**Sinusoidal (Periodic)**

e.g., Seasonal Sales, Temperatures



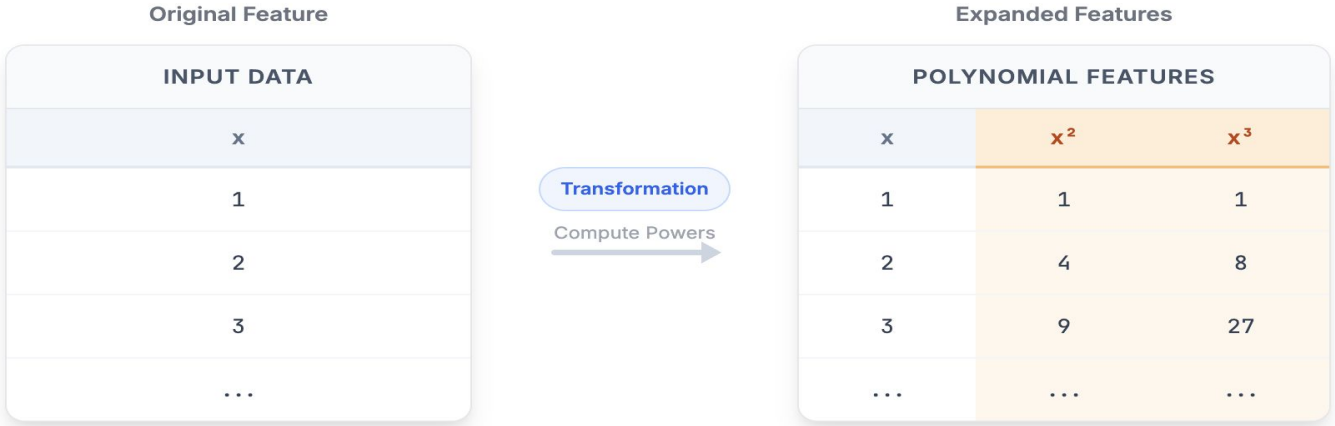
**Exponential Growth**

e.g., Bacterial Growth, Virality

- Real-world data is rarely perfectly straight lines. We need flexibility.
- Linear models fail to capture these fundamental relationships
- The model is too simple to capture the underlying pattern or curvature.

# Polynomial Regression

## The Idea: Feature Transformation




THE MODEL

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$

- We don't change the algorithm - we change the data! This is called feature engineering.
- Curvature comes from the transformed features, allowing complex fits.
- Still using Linear Regression, just on transformed features!
- Higher powers allow the model to capture curves and bends

# Polynomial Regression

## How It Works

- Input x; D=2 ⇒ $x, x^2; w_0 + w_1x + w_2x^2$
- Input x; D=3 ⇒ $x, x^2, x^3$
- Input $x_1, x_2$; D=2 ⇒ $x_1, x_2, x_1^2, x_2^2, x_1x_2$
- Input $x_1, x_2$; D=3 ⇒ $x_1, x_2, x_1^2, x_2^2, x_1x_2, x_1^3, x_2^3, x_1^2x_2, x_1x_2^2$

**Formula for Feature Growth**

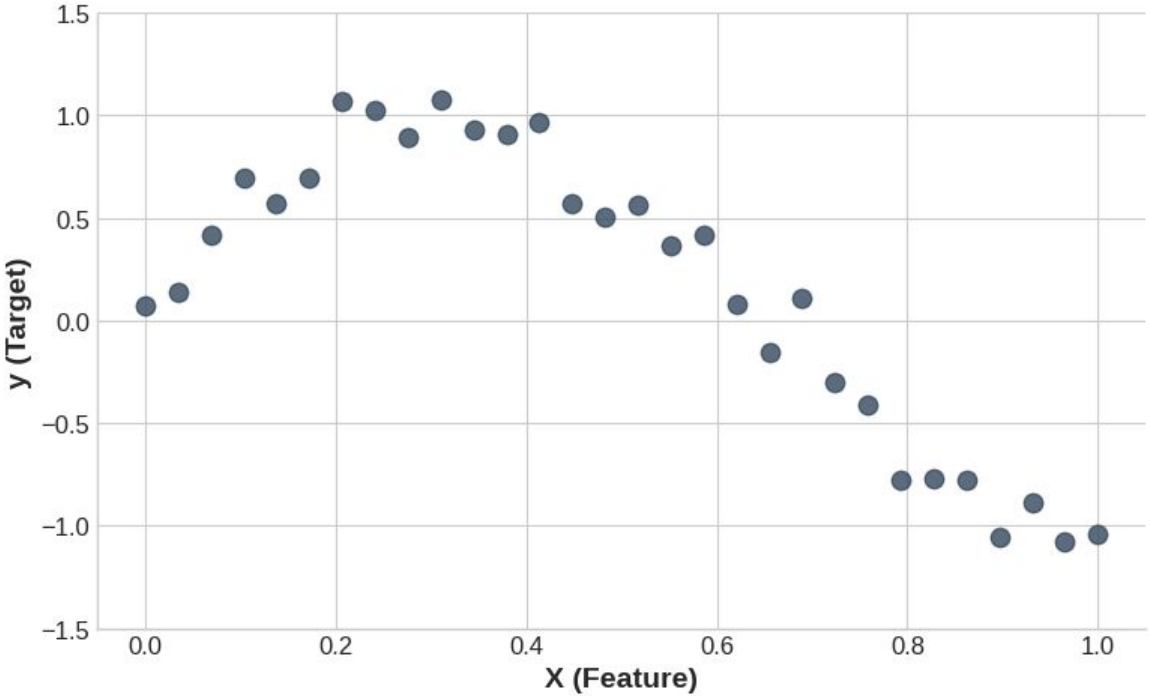
$$C(p+d, d) = (p+d)! / (p! \times d!)$$

where p = original features, d = degree

| Original | Degree | Total |
|---|---|---|
| 3 | 2 | 9 |
| 3 | 3 | 19 |
| 5 | 3 | 55 |
| 10 | 3 | 285 |

# Polynomial Regression

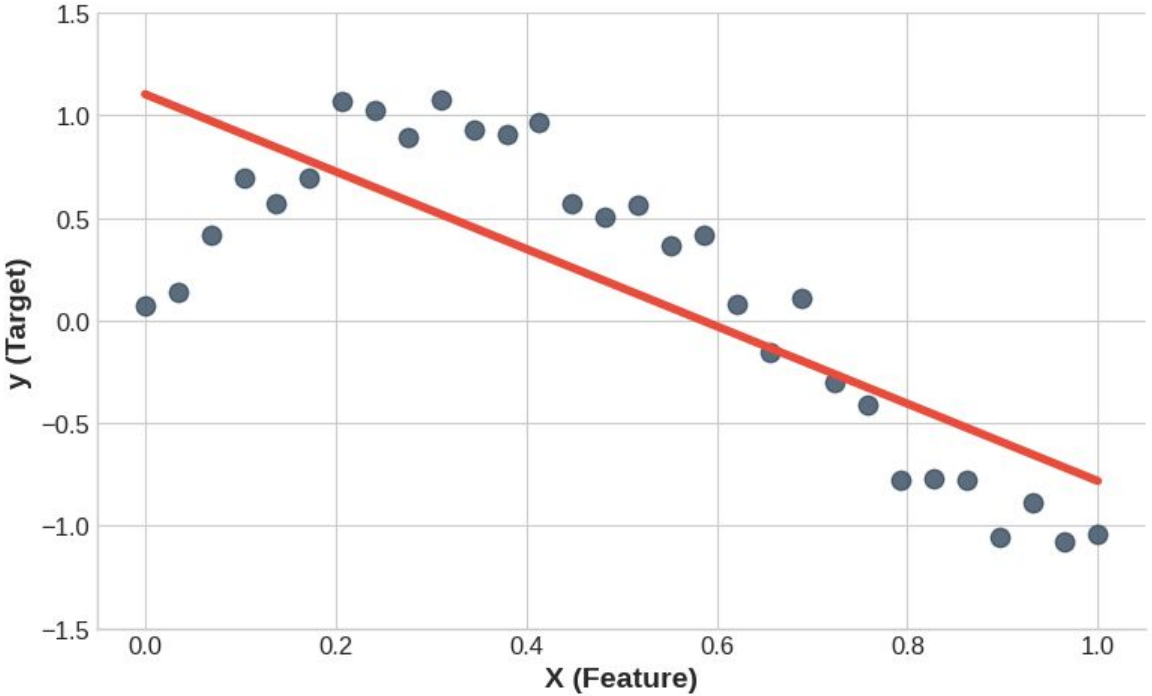
## Sinusoidal Data: f(x) = sin(πx) + ε




**What degree of polynomial should I use?**

# Polynomial Regression

## Degree 1 (Linear)

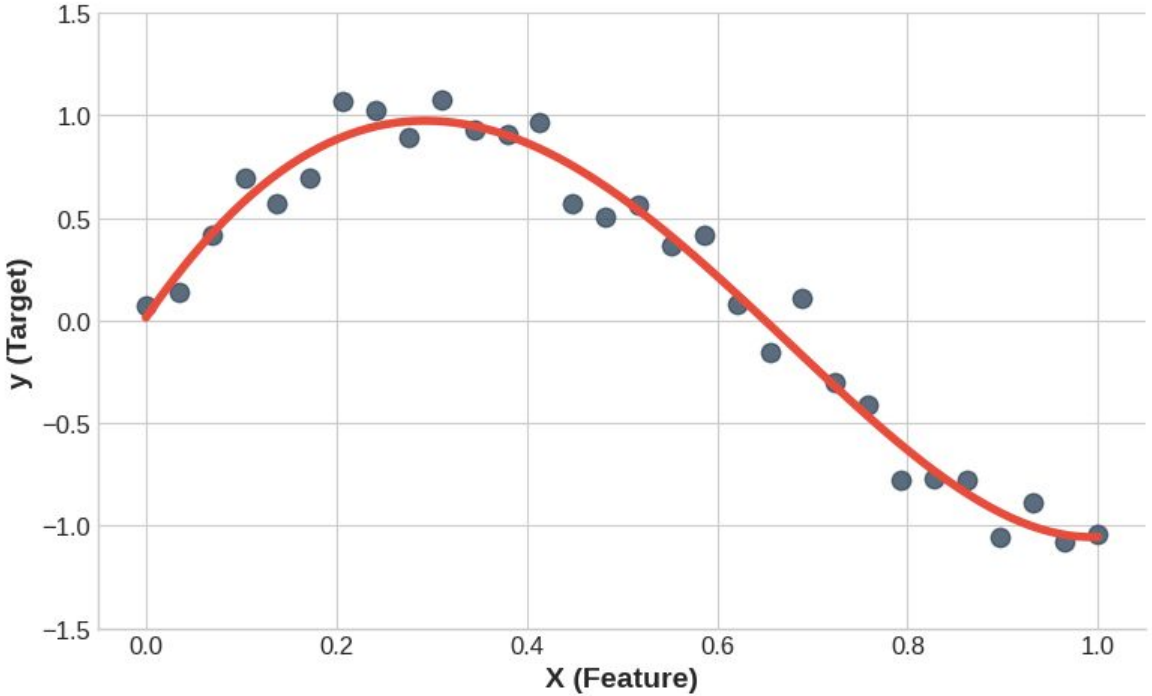



**What degree of polynomial should I use?**

- Degree = 1?
- $R^2 = 0.05$

# Polynomial Regression

## Degree 4

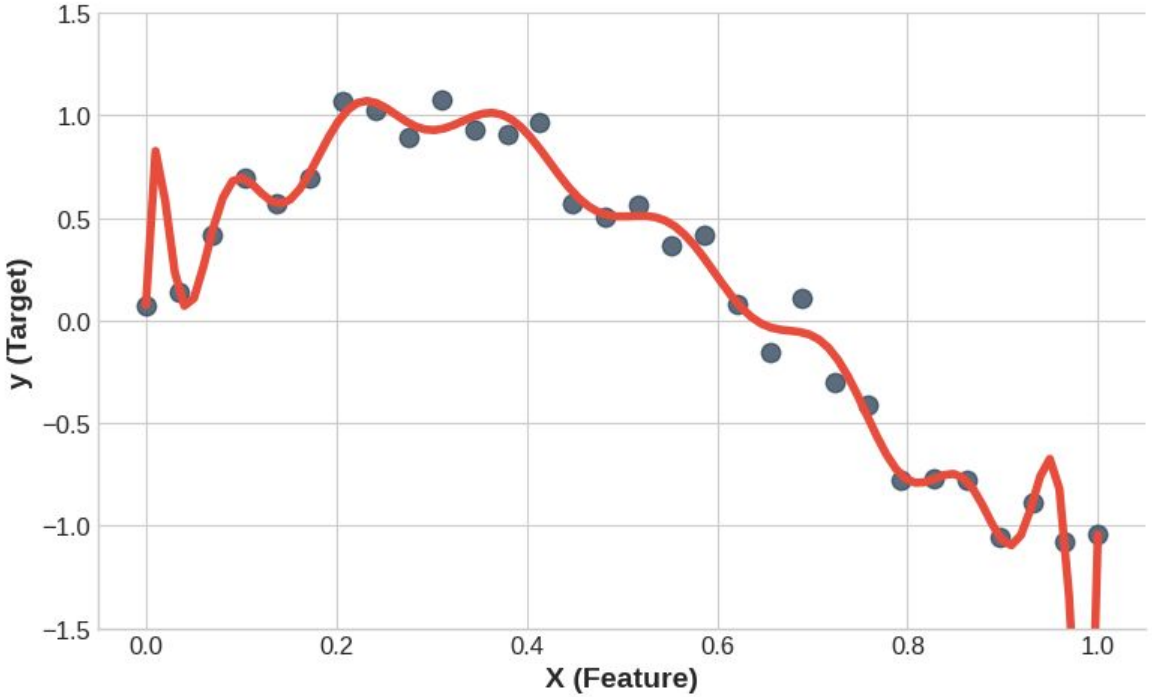



**What degree of polynomial should I use?**

- Degree = 4?
- $R^2 = 0.85$

# Polynomial Regression

## Degree 15




**What degree of polynomial should I use?**

- Degree = 15?
- $R^2 = 0.98$

# Overfitting and Underfitting

Training Performance: Low
Testing Performance: Low

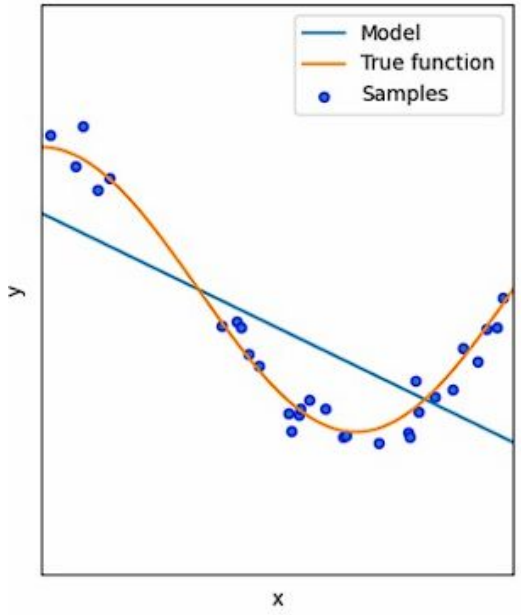



Fig 01: Underfitting

Training Performance: Good
Testing Performance: Good

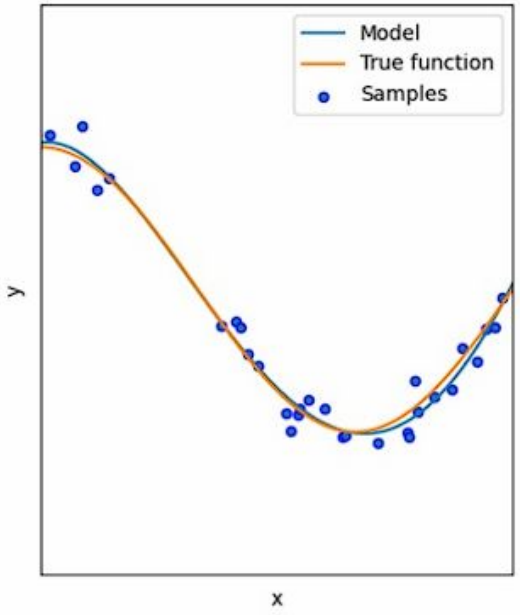



Fig 02: Best fitting

Training Performance: High
Testing Performance: Low

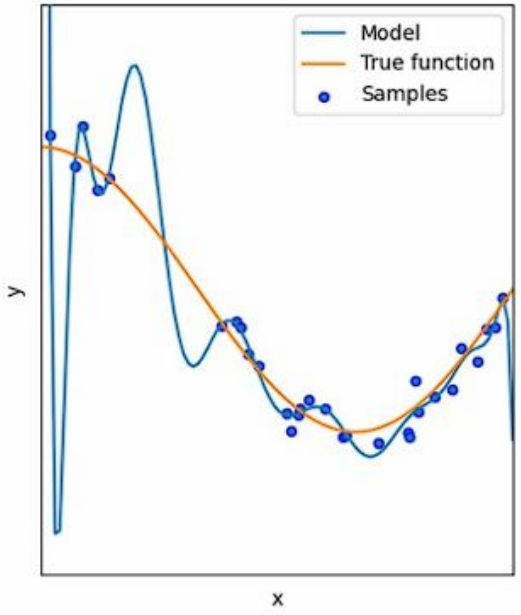
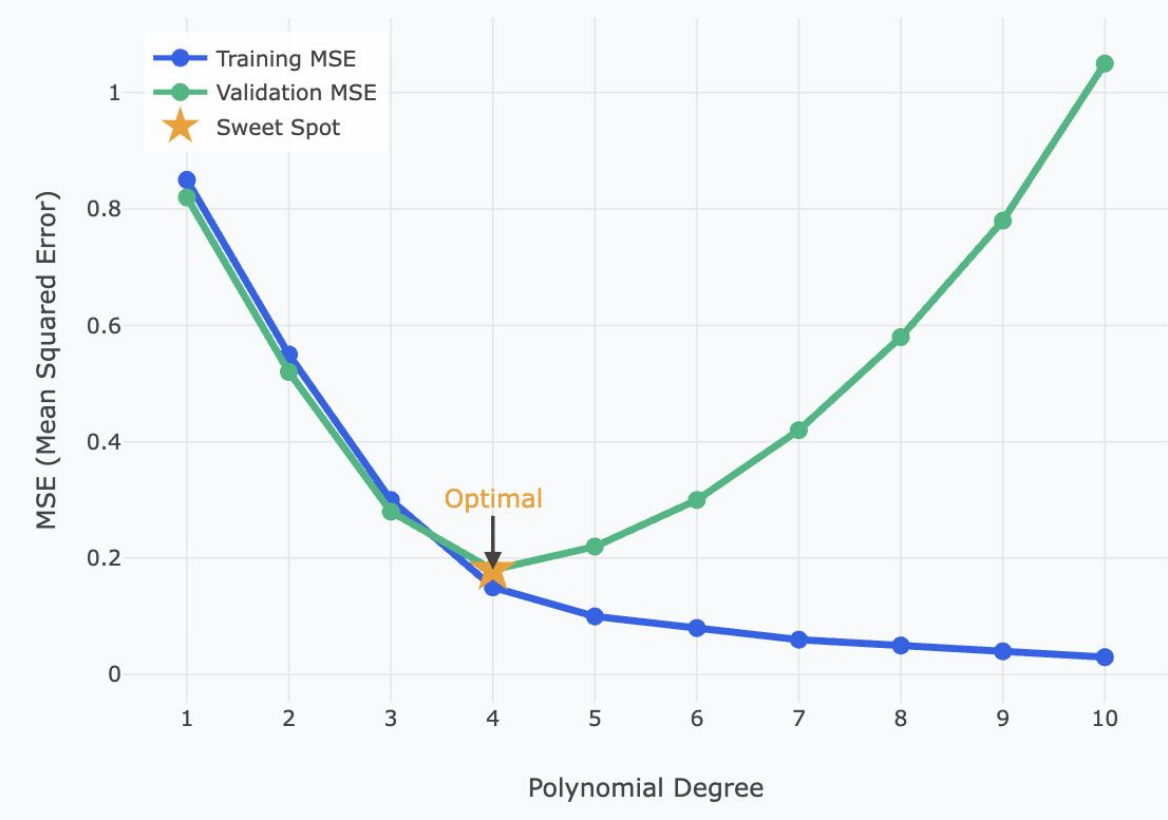


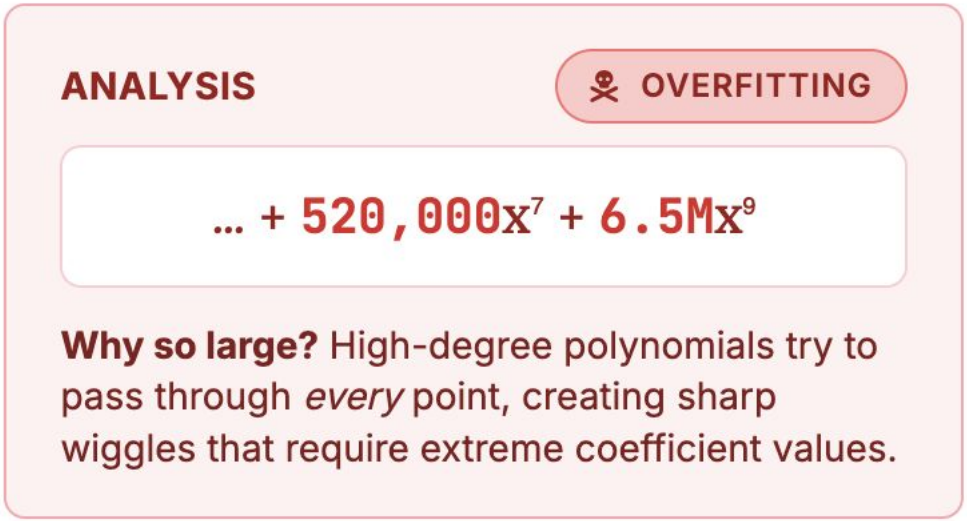

Fig 03: Overfitting

# Training vs Validation Performance



# What Causes the Wiggles?

- Why does a high-degree polynomial start oscillating wildly between data points?




- To pass through every noisy point, the curve must make sharp turns.
- Sharp turns require EXTREME coefficient values.
- Large coefficients mean the model is unstable. A tiny change in input (x) gets multiplied by a huge number (w), causing a massive change in prediction (ŷ).

# Regularization: Add a Penalty to the Loss

- Regularization is a technique used in machine learning to prevent overfitting and improve the generalization of a model. Overfitting occurs when a model learns the training data too well, including its noise and outliers, to the extent that it performs poorly on new, unseen data.
- Goal 1: Fit the Data (Minimize MSE)
- Goal 2: Keep Model Simple (Penalize Large Coeffs)
- LOSS = MSE + $\alpha \times$ (penalty)
- Alpha ($\alpha$) is a hyperparameter. The model doesn't learn it. YOU must choose it by testing on Validation Data.
- $\alpha = 0$, no regularization, risk of overfitting.
- $\alpha$ = large value, strong regularization, Coefficients $\approx 0$, risk of underfitting.
- Regularization methods add a penalty term to the model's loss function, discouraging the learning algorithm from assigning excessive importance to any one feature or allowing the model to become too complex.

# Regularization

## Lasso (L1)

Lasso (Least Absolute Shrinkage and Selection Operator) adds a penalty equal to the **sum of absolute values** of the coefficients (L1 norm) to the loss function.

$$\text{Loss} = \text{Base Loss} + \alpha \sum_{j=1}^{p} |w_j|$$

Where:

- **Base Loss** = MSE (or another regression loss function)
- $\alpha$ = Regularization strength
- $\sum |w_j|$ = L1 penalty term

- Shrinks coefficients toward zero. Can make some coefficients exactly zero.
- Performs feature selection.
- L1 pushes unimportant features completely out of the model.

# Regularization

## Ridge (L2)

Ridge adds an L2 penalty (sum of squared coefficients) to the loss function.

$$\text{Loss} = \text{Base Loss} + \alpha \sum_{j=1}^{p} w_j^2$$

Where:

- **Base Loss** = MSE (or any regression loss)
- $\alpha$ = Regularization strength
- $\sum w_j^2$ = L2 penalty term

- Shrinks coefficients toward zero.
- Does not make coefficients exactly zero.
- Keeps all features in the model

# Ridge vs Lasso

*Same problem. Different behavior on coefficients.*

| Feature | Ridge Coefficient | Lasso Coefficient |
|---|---|---|
| x | 0.83 | 0.79 |
| $x^2$ | 0.62 | 0.55 |
| $x^3$ | 0.31 | 0.00 |
| $x^4$ | 0.18 | 0.00 |
| noise_1 | 0.07 | 0.00 |
| noise_2 | 0.04 | 0.00 |

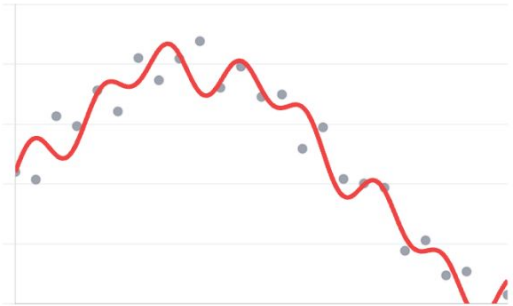
*Lasso zeroed out $x^3$, $x^4$, noise_1, noise_2 entirely. Ridge kept all of them, just smaller. Both are right for different situations.*

# Effect of Alpha (α) Strength

## Same degree polynomial (degree 9) with different α values

α = 0.001

**TOO WEAK**



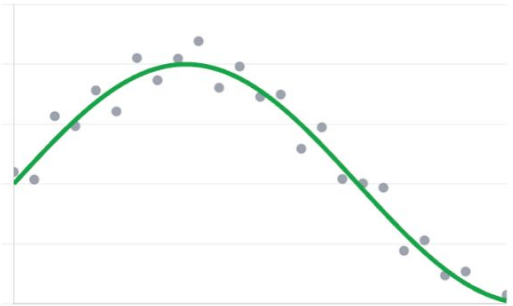
**Problem: Overfitting**

The penalty is too small. The model is free to wiggle and chase noise. Coefficients explode.

`High Variance`

α = 0.1

**JUST RIGHT**



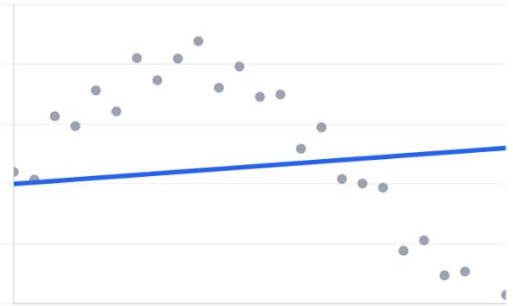
**Result: Good Fit**

The penalty is balanced. It smooths out the wiggles but keeps the underlying curve structure.

`Good Generalization`

α = 10.0

**TOO STRONG**



**Problem: Underfitting**

The penalty is massive. It forces all coefficients near zero, turning the model into a straight line.

`High Bias`

# Feature Impact Understanding

## Reading Coefficients

*The coefficient tells you the direction and relative strength of each feature*

**+**

### Positive coefficient

As this feature increases, the prediction increases. The larger the value, the stronger the effect.

**-**

### Negative coefficient

As this feature increases, the prediction decreases. Same magnitude logic applies.

**0**

### Near-zero coefficient

This feature has almost no influence on the output. Lasso may push it all the way to zero.

**↔**

### Large absolute value

Dominant feature in the model. A unit change here produces a large change in prediction.

Always scale your features before comparing coefficients. Unscaled features with large values will always look dominant, even if they are not.

# How to implement in Python?

# sklearn Implementation - Polynomial Features

```python
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Step 1: Create polynomial features
poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)
# X_poly: ['1', 'x0', 'x0^2', 'x0^3']

# Step 2: Train linear regression on polynomial features
model = LinearRegression()
model.fit(X_poly, y)

# Step 3: Predict on new data
X_new_poly = poly.transform(X_new)
predictions = model.predict(X_new_poly)
```

# sklearn Implementation - Ridge and Lasso

```python
# Ridge (L2)
from sklearn.linear_model import Ridge

model = Ridge(alpha=0.1)
model.fit(X_poly, y)
```

```python
# Lasso (L1)
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
model.fit(X_poly, y)
```

# Hyperparameter Tuning with GridSearchCV

```python
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

# Create pipeline
pipeline = Pipeline([
    ('poly', PolynomialFeatures()),
    ('ridge', Ridge())
])

# Define grid
param_grid = {
    'poly__degree': [2, 3, 4],
    'ridge__alpha': [0.1, 1, 10]
}

# Find best combination
grid = GridSearchCV(pipeline, param_grid, cv=5)
grid.fit(X_train, y_train)

print(grid.best_params_)
# {'poly__degree': 3, 'ridge__alpha': 1}
```


Q&A